RISKGUARD – A MULTI-MODAL AI DEEPFAKE DETECTION SYSTEM

A PROJECT REPORT

Submitted by

ABI PRASATH R	**730422205002**
HARIJA A	**730422205023**
HEMABALA S P	**730422205030**

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

ERODE SENGUNTHAR ENGINEERING COLLEGE
(AUTONOMOUS)
PERUNDURAI, ERODE – 638057

APRIL 2026

ERODE SENGUNTHAR ENGINEERING COLLEGE (AUTONOMOUS) PERUNDURAI, ERODE-638057.

BONAFIDE CERTIFICATE

Certified that this project report **"RISKGUARD – A MULTI-MODAL AI DEEPFAKE DETECTION SYSTEM"** is the Bonafide work of **ABI PRASATH R (730422205002), HARIJA A (730422205023)** and **HEMABALA S P (730422205030)** who carried out the project work under my supervision. Certified further that to the best of knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE
Mr.R.NARENDRAN,B.E.,M.E.,(Ph.D).,
SUPERVISOR
Assistant Professor,
Department of Information Technology,
Erode Sengunthar Engineering College,
Perundurai,
Erode - 638 057.

SIGNATURE
Dr.M.VIJAYAKUMAR,M.E.,Ph.D.,
Professor and Dean,
Department of Information Technology,
Erode Sengunthar Engineering College,
Perundurai,
Erode - 638 057.

Submitted for the End Semester Project Work Viva-Voce examination held on _____ for **19IT801 - Project Work** during the academic year 2025-2026.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

First of all, we thank the almighty God for the blessings and support given to us to complete this project successfully.

We would like to express our deep thanks to our beloved President **Thiru V. ANNADURAI** & Correspondent **Thiru G. KAMALAMURUGAN, BBM.,** for giving motivation and providing all necessary facilities for successful completion of the project.

We take this opportunity to express our profound gratitude and deep regards to **Thiru S. N. THANGARAJU, B.E., MBA.,** Secretary, for his constant encouragement throughout the course of our work.

With pleasure, we would like to express our gratefulness to **Dr. V. VENKATACHALAM, B.E., M.S., M.Tech., Ph.D.,** Principal, for the moral support and deeds in bringing out this project successfully.

We express our gratitude to the Dean, Department of Information Technology **Dr. M. VIJAYAKUMAR, M.E., Ph.D.,** for encouraging a lot throughout this work.

We would like to express our thanks and sincere gratitude to our project coordinator **Mr. T. PRASANTH, B.E., M.E.,(Ph.D).,** Assistant Professor and our Guide **Mr. R. NARENDRAN, B.E., M.E., (Ph.D).,** Assistant Professor, from the Department of Information Technology, for their invaluable support and guidance throughout the project.

We express our thanks to all our Teaching and Non-Teaching staff members for their advice and encouragement to do the project work with full interest and enthusiasm.

Finally, we would like to thank all our friends and family members for providing us a constant support to complete this project work.

ABSTRACT

The rapid proliferation of AI-generated media including deepfake images produced by Stable Diffusion and DALL-E, cloned voices synthesised by XTTS and VITS, machine-generated phishing text authored by GPT-4, and face-swapped videos poses serious and escalating challenges to digital trust, cybercrime investigation, and public safety. Existing detection tools require manual content submission and are limited to single modalities, leaving citizens unprotected during everyday application usage. RiskGuard addresses these challenges through a comprehensive, production-grade, multi-modal detection platform that identifies synthetic content across four modalities and preserves forensic evidence on a public blockchain.The system employs a hybrid CPU + GPU architecture combining lightweight, zero-dependency local signal processing with cloud-based transformer models. The image detection engine uses a 6-signal type-adaptive weighted ensemble incorporating Noise Print Residual analysis, DCT spectral fingerprinting, Haar wavelet decomposition, perceptual hashing, and cloud-based CNN and HuggingFace classifiers. A structural classifier dynamically adjusts signal weights based on image type (photograph, digital art, or screenshot), preventing false positives on non-photographic content. The voice detection pipeline uses a 6-signal hybrid architecture with LFCC, CQT phase coherence, modulation spectrum analysis, pitch contour tracking, statistical moment features, and a wav2vec2-base model fine-tuned on ASVspoof2019 data. Text detection combines DeBERTa and RoBERTa cloud classifiers with local statistical analysis and a Binoculars perplexity proxy for zero-shot LLM detection.To ensure forensic integrity, detected evidence is secured using SHA-256 hashing, IPFS decentralised storage via Pinata, and Merkle-tree batch anchoring on the Polygon Amoy blockchain through a custom EvidenceAnchor smart contract. Merkle batching reduces per-record blockchain costs by up to 99.7% compared to individual transactions. A Flutter mobile application serves public users with real-time protection through a system overlay that monitors whitelisted applications (WhatsApp, Chrome, Instagram, Telegram, Facebook) for deepfake content, while a FastAPI-powered web dashboard provides law enforcement investigators with real-time, tamper-proof evidence review capabilities through Server-Sent Events. Experimental results demonstrate 70–92% accuracy on image detection, 65–85% on voice detection, and 70–88% on text detection, with real-time processing under 200ms per voice chunk and under 150ms for local image analysis.

Keywords: Deepfake Detection, AI-Generated Content, Multi-Modal Analysis, Blockchain Evidence, Merkle Trees, IPFS, Flutter, FastAPI, Real-Time Protection Overlay, Edge Computing, Cybersecurity.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	TABLE OF CONTENTS	**v**
	LIST OF FIGURES	**vii**
	LIST OF TABLES	**viii**
	LIST OF ABBREVIATIONS	**ix**
1	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM STATEMENT	1
	1.3 CONTRIBUTIONS	2
2	**LITERATURE SURVEY**	**3**
3	**SYSTEM ANALYSIS**	**7**
	3.1 EXISTING SYSTEM	7
	3.1.1 LIMITATIONS OF EXISTING SYSTEM	8
	3.2 PROPOSED SYSTEM	8
	3.2.1 ADVANTAGE OF PROPOSED SYSTEM	9
4	**SYSTEM SPECIFICATION**	**10**
	4.1 HARDWARE SPECIFICATION	10
	4.2 SOFTWARE SPECIFICATION	10

5	**PROJECT DESCRIPTION**	**12**
	5.1 SYSTEM ARCHITECTURE	12
	5.2 LIST OF MODULES	13
	5.3 MODULE DESCRIPTION	13
	5.4 WORKING OF SYSTEM	22
6	**TESTING AND EVALUATION**	**24**
	6.1 TYPES OF TEST	24
	6.1.1 UNIT TESTING	24
	6.1.2 INTEGRATION TESTING	24
	6.1.3 SYSTEM TESTING	25
	6.1.4 USER ACCEPTANCE TESTING	25
	6.2 TEST CASE DESIGN	26
7	**CONCLUSION AND FUTURE SCOPE**	**27**
	7.1 CONCLUSION	27
	7.2 FUTURE SCOPE	27
8	**APPENDIX-I**	**28**
	SOURCE CODE	
9	**APPENDIX-II**	**38**
	SCREENSHOTS	
10	**REFERENCES**	**43**

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
5.1	High-Level System Architecture	12
5.2	Image Analysis Pipeline Flow	15
5.3	Voice Analysis Signal Architecture	17
5.4	Text Analysis Ensemble Flow	18
5.5	Blockchain Evidence Pipeline	19
5.6	Real-Time Protection Overlay Flow	20
9.1	Mobile Application Home Screen	38
9.2	Real-Time Protection Overlay on Social Media	38
9.3	Voice Deepfake Detection Interface	39
9.4	Image Analysis - 6-Signal Ensemble Results	39
9.5	Blockchain Evidence Filing Confirmation	40
9.6	LLM Text Detection and Phishing Analysis	40
9.7	Cybercrime Investigation Web Dashboard	41
9.8	Call Monitoring Overlay During Active Call	41
9.9	FastAPI Backend Server Request Handling	42
9.10	Solidity Smart Contract on Remix IDE	42

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
4.1	Technology Stack and Framework Components	10
5.1	Image Detection Signal Weights by Content Type	14
5.2	Voice Detection Signal Weights and Techniques	16
5.3	Real-Time Overlay Risk Scoring Components	21
6.1	System Test Cases and Validation Results	26

LIST OF ABBREVIATIONS

AI	**A**rtificial **I**ntelligence
ML	**M**achine **L**earning
CNN	**C**onvolutional **N**eural **N**etwork
LLM	**L**arge **L**anguage **M**odel
ViTS	**V**ision **T**ransformer
DCT	**D**iscrete **C**osine **T**ransform
LFCC	**L**inear **F**requency **C**epstral **C**oefficients
MFCC	**M**el-**F**requency **C**epstral **C**oefficients
CQT	**C**onstant-**Q** **T**ransform
API	**A**pplication **P**rogramming **I**nterface
NLP	**N**atural **L**anguage **P**rocessing
SSE	**S**erver-**S**ent **E**vents
ONNX	**O**pen **N**eural **N**etwork **E**xchange
TTS	**T**ext-**t**o-**S**peech
GAN	**G**enerative **A**dversarial **N**etwork
URL	**U**niform **R**esource **L**ocator
XTTS	**C**ross-lingual **T**ext-**t**o-**S**peech
VITS	**V**ariational **I**nference **T**ext-to-**S**peech
LLM	**L**arge **L**anguage **M**odel
IPFS	**I**nter **P**lanetary **F**ile **S**ystem

CPU	**C**entral **P**rocessing **U**nit
GPU	**G**raphics **P**rocessing **U**nit
GPT - 4	**G**enerative **P**re-trained **T**ransformer **4**
HTTPS	**H**yper **T**ext **T**ransfer **P**rotocol **S**ecure
HF	**H**ugging **F**ace (AI platform/company)
NPR	**N**oise **P**rint **R**esidual
SHA-256	**S**ecure **H**ash **A**lgorithm 256-bit
VAD	**V**oice **A**ctivity **D**etection

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

The proliferation of AI-generated media—such as deepfake images created using Stable Diffusion, cloned voices via XTTS/VITS, and AI-authored phishing text powered by GPT-4 poses an escalating threat to digital trust, cybercrime investigation, and public safety. The rapid advancement of generative AI technologies has made it increasingly difficult for both humans and traditional software systems to distinguish authentic content from synthetic media. This has created unprecedented challenges for law enforcement agencies, financial institutions, and ordinary citizens who depend on the reliability and authenticity of digital communications.

RiskGuard addresses this growing threat landscape by offering a comprehensive multi-modal detection platform that integrates local signal processing with cloud-based AI models. The system is designed as a dual-interface solution, featuring a Flutter-based mobile application for citizen-facing threat detection with real-time protection overlay, alongside a FastAPI-powered web dashboard for law enforcement evidence management. This hybrid architecture ensures accessibility and performance even on commodity hardware, while also maintaining forensic-grade evidence integrity through blockchain anchoring.

1.2 PROBLEM STATEMENT

Law enforcement agencies face four core challenges in the current digital landscape. The first is detection, which involves accurately distinguishing AI-generated content from authentic media across multiple modalities such as images, voice, text, and video, while maintaining low latency. The second challenge is evidence preservation, where agencies must ensure a tamper-proof chain of custody for digital forensic evidence that can withstand legal scrutiny in courts. These challenges are further complicated by the increasing sophistication of generative AI technologies, which make synthetic content more realistic and harder to identify.

In addition to detection and preservation, accessibility and real-time protection remain critical concerns. There is a growing need to provide real-time detection tools that are easily accessible to both citizens and investigators, enabling proactive identification of threats without requiring manual initiation. However, most existing solutions rely on users to manually submit

suspicious content for analysis, creating a significant gap in security. This limitation results in many threats going undetected during regular app usage, highlighting the urgent need for systems that offer continuous, automated protection.

1.3 CONTRIBUTIONS

RiskGuard addresses the key challenges in detecting AI-generated threats through a robust multi-signal ensemble architecture applied across different data modalities. This approach achieves an accuracy of 60–92% while remaining efficient enough to operate without GPU hardware on the detection path, making it suitable for deployment on commodity devices. Additionally, the system incorporates a blockchain-based evidence ledger, known as the Nimirdhu Nill Threat Ledger, which anchors batches of forensic evidence onto a public blockchain using Merkle tree structures. This design significantly reduces transaction costs, minimizing gas fees by up to 99.7% while preserving data integrity and auditability.

Furthermore, RiskGuard adopts a dual-interface design to serve both citizens and investigators effectively. A Flutter-based mobile application enables real-time threat detection and reporting for end users, while a FastAPI-powered web dashboard supports law enforcement with advanced evidence management features and real-time notifications via Server-Sent Events. Complementing these capabilities, the system includes a proactive real-time protection overlay that continuously monitors whitelisted applications and automatically scans URLs, images, videos, and voice calls without requiring user intervention, ensuring seamless and continuous security.

CHAPTER 2

LITERATURE SURVEY

2.1 Noiseprint: A CNN-Based Camera Model Fingerprint

Cozzolino & Verdoliva (2020)

This foundational work in camera model fingerprinting introduces an advanced method for identifying the source of digital images by extracting intrinsic sensor noise patterns. The authors propose the concept of a Noise Print Residual (NPR), which is obtained by computing the difference between an original image and its median-filtered version. This process effectively isolates camera-specific noise artifacts that are typically invisible to the human eye but remain consistent across images captured by the same device. These unique noise patterns serve as reliable fingerprints for distinguishing between images from different cameras.

AI-generated images, particularly those produced by modern generative models, lack these natural sensor noise characteristics, making NPR a powerful tool for detection. In the RiskGuard system, this technique is adapted as one of the key signals in the image detection ensemble. The system analyzes features such as noise variance, block consistency, and spectral flatness to differentiate authentic photographs from synthetic images. This approach significantly enhances the robustness of image-based deepfake detection.

2.2 End-to-End Anti-Spoofing with RawNet2

Tak et al. (2021)

This research presents a novel end-to-end framework for detecting spoofed audio using raw waveform inputs rather than traditional handcrafted features. The study highlights the effectiveness of Constant-Q Transform (CQT) in analyzing phase coherence, which reveals subtle artifacts introduced by neural vocoders in synthesized speech. Unlike conventional Short-Time Fourier Transform (STFT) representations, CQT provides a more detailed frequency resolution, making it suitable for detecting inconsistencies in generated audio signals.

RiskGuard incorporates this concept by utilizing CQT-based phase coherence analysis within its voice detection pipeline. Specifically, it measures phase derivative variance to identify

unnaturally consistent phase patterns that are characteristic of text-to-speech and voice cloning systems. This method allows the system to detect even highly realistic synthetic audio, thereby improving the accuracy and reliability of voice deepfake detection.

2.3 wav2vec 2.0: Self-Supervised Learning of Speech Representations Baevski et al. (2020)

This seminal work introduces wav2vec 2.0, a self-supervised learning framework designed to extract meaningful speech representations from large amounts of unlabeled audio data. The model leverages contrastive learning techniques to capture contextual information in speech signals, making it highly effective for a wide range of speech processing tasks. Its ability to learn robust features without requiring extensive labeled datasets has made it a widely adopted standard in speech recognition and anti-spoofing applications.

In the RiskGuard system, a wav2vec2-base model is fine-tuned using the ASVspoof2019 dataset to enhance its capability in detecting synthetic speech. The model is deployed via ONNX export on a GPU-enabled environment, contributing a significant weighted signal to the voice detection framework. This cloud-based component complements multiple lightweight CPU-based signals, ensuring a balanced and efficient approach to deepfake voice detection.

2.4 Spotting LLMs with Binoculars Hans et al. (2024)

This recent study proposes a zero-shot detection approach for identifying AI-generated text by comparing perplexity scores derived from two different language model perspectives. The method leverages differences in probability distributions to distinguish between human-written and machine-generated text without requiring task-specific training. This makes it a flexible and generalizable solution for detecting outputs from various large language models.

RiskGuard implements a simplified version of this concept using unigram and bigram entropy ratios as proxy measures for perplexity. This lightweight approach enables efficient detection of AI-generated text without the need for computationally intensive models. It acts as a universal

signal that complements more specialized classifiers such as DeBERTa and RoBERTa, thereby strengthening the overall text detection ensemble.

2.5 Design, Analysis and Experimental Evaluation of Block Level Features Sahidullah et al. (2015)

This work provides a comprehensive analysis of statistical features for detecting spoofed speech, emphasizing the importance of higher-order amplitude moments such as kurtosis and skewness. The study demonstrates that these statistical properties differ significantly between natural and synthetic audio signals due to variations in signal generation processes. These features serve as effective indicators for identifying anomalies in speech data.

RiskGuard incorporates these statistical measures as part of its voice detection pipeline to enhance robustness. By analyzing amplitude distributions and identifying deviations from natural speech patterns, the system can detect synthetic audio even when advanced detection methods are unavailable. This lightweight feature set acts as a reliable backup signal, ensuring consistent performance across different scenarios.

2.6 ASVspoof 5: Crowdsourced Speech Synthesis Detection. ISCA Interspeech ASVspoof Consortium (2024).

The ASVspoof 5 challenge represents a significant advancement in the field of automatic speaker verification (ASV) and anti-spoofing research. Organized by the ASVspoof Consortium, this edition introduces a crowdsourced approach to speech synthesis detection, incorporating a diverse range of spoofing attacks generated using modern text-to-speech (TTS) and voice conversion techniques. The dataset includes realistic and varied audio samples collected from multiple sources, making it more representative of real-world scenarios. This diversity enhances the robustness of evaluation frameworks and encourages the development of generalized anti-spoofing models capable of detecting unseen attacks.

In the RiskGuard system, insights from ASVspoof 5 are utilized to improve the effectiveness of voice deepfake detection. The system leverages similar evaluation strategies and incorporates training data inspired by such benchmark datasets to enhance model generalization. By aligning with the methodologies proposed in ASVspoof 5, RiskGuard ensures that its detection pipeline

remains resilient against evolving speech synthesis technologies. This contributes to improved accuracy and reliability in identifying spoofed or AI-generated audio in practical applications.

2.7 IPFS – Content Addressed, Versioned, P2P File System. arXiv:1407.3561 Benet, J. (2014).

This work introduces the InterPlanetary File System (IPFS), a decentralized peer-to-peer file storage system designed to make the web faster, safer, and more open. Unlike traditional HTTP-based systems that rely on location-based addressing, IPFS uses content addressing, where each file is identified by a unique cryptographic hash. This ensures data integrity, as any modification to the file results in a completely different hash. Additionally, IPFS supports versioning and distributed storage, allowing files to be stored across multiple nodes, thereby improving redundancy and availability while reducing dependency on centralized servers.

In the RiskGuard system, IPFS concepts are utilized for secure and efficient storage of forensic evidence such as images, audio files, and analysis reports. By storing content using hash-based identifiers, the system ensures tamper-proof data management and easy verification of file authenticity. This approach also complements blockchain integration, where only the hash references are stored on-chain, significantly reducing storage costs while maintaining traceability and integrity. As a result, RiskGuard achieves a scalable and secure architecture for handling large volumes of digital evidence.

CHAPTER 3
SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Existing System: Manual Deepfake Detection Tools:

Overview

Current deepfake detection solutions such as Microsoft Video Authenticator, Deepware Scanner, and Sensity AI operate as standalone analysis tools that require users to manually upload suspicious content for verification. These systems typically focus on a single modality (usually image or video) and provide results after significant processing delays.

Working Mechanism

- Uses pre-trained CNN models for face manipulation detection.
- Requires manual file upload or URL submission.
- Processes content in batch mode with results returned asynchronously.
- Limited to single-modality analysis (image OR video OR audio).

Technologies Used

- Deep Learning frameworks (TensorFlow, PyTorch).
- Cloud-based GPU inference.
- Web-based interfaces.

Advantages

- High accuracy on specific deepfake types.
- Utilises state-of-the-art transformer models.

Existing System: Blockchain-Based Digital Evidence Systems:

Overview

Systems like Chainkit and Amber Authenticate use blockchain technology to verify the authenticity and provenance of digital files. These systems focus on evidence integrity but lack integrated AI detection capabilities, requiring separate tools for content analysis.

Working Mechanism

- Cryptographic hashing of digital files.
- On-chain storage of file hashes for tamper detection.
- Provenance tracking through blockchain transactions.

Advantages

- Tamper-proof evidence chain.
- Immutable audit trail.

3.1.1 LIMITATIONS OF EXISTING SYSTEM

Both categories of existing systems suffer from several critical limitations. Manual deepfake detection tools require user-initiated analysis, meaning threats encountered during normal app usage go undetected. They are restricted to single-modality analysis, unable to cross-correlate signals across image, voice, text, and video simultaneously. Processing is typically slow (10–60 seconds per item) and requires cloud connectivity, making offline detection impossible. Blockchain evidence systems operate independently of detection, creating a fragmented workflow where investigators must use separate tools for detection and evidence preservation. Neither approach provides real-time, proactive protection during everyday app usage such as browsing WhatsApp, Chrome, or Instagram.

3.2 PROPOSED SYSTEM

The proposed system, RiskGuard, is a comprehensive multi-modal AI-generated content detection platform that integrates detection, evidence preservation, and real-time protection into a single unified system. Unlike existing solutions that require manual content submission, RiskGuard features a proactive real-time protection overlay that automatically monitors whitelisted applications and scans content as users encounter it during normal usage.

RiskGuard utilises a hybrid CPU + GPU architecture combining six-signal ensemble detection for images, six-signal detection for voice, four-signal detection for text, and two-signal temporal-aware detection for video. The system operates on commodity 8 GB RAM hardware with optional cloud GPU acceleration via HuggingFace Inference API and Google Colab ONNX endpoints. All detection results can be filed as evidence with cryptographic proof trails using SHA-256 hashing, IPFS decentralised storage, and Merkle-batch anchoring to the Polygon Amoy blockchain.

3.2.1 ADVANTAGE OF PROPOSED SYSTEM

- Multi-modal detection across image, voice, text, and video in a single platform.
- Real-time proactive protection through a system overlay that monitors apps automatically.
- Hybrid local+cloud architecture enabling offline detection with optional cloud enhancement.
- Integrated blockchain evidence chain with Merkle-batch anchoring reducing gas costs by 99.7%.
- Dual interface serving both citizens (Flutter app) and investigators (web dashboard).
- Live call monitoring with voice deepfake detection during active phone calls.
- URL verification with threat intelligence from URLhaus and community blacklists.
- Processing performance under 200ms for real-time voice chunks and under 150ms for local image analysis.

CHAPTER 4
SYSTEM SPECIFICATION

4.1 HARDWARE SPECIFICATION

The RiskGuard system requires a standard hardware setup to support AI processing, backend operations, and mobile application deployment. The development environment includes a minimum of 8 GB RAM for running the FastAPI backend with concurrent model inference. Storage requirements include a minimum of 256 GB SSD for faster model loading and evidence storage. An Android mobile device running API level 26+ (Android 8.0 Oreo) or higher is required for the Flutter mobile application with overlay support. The overlay system requires the device to support the SYSTEM_ALERT_WINDOW permission and accessibility services for real-time content monitoring.

For AI and deep learning operations, a dedicated GPU (such as NVIDIA GTX/RTX series) is optional but beneficial for local model training. In production deployment, GPU inference is offloaded to Google Colab endpoints running ONNX-exported models, eliminating the need for local GPU hardware. A stable internet connection is essential for HuggingFace API inference, IPFS evidence storage, blockchain transactions, and URL verification via URLhaus.

4.2 SOFTWARE SPECIFICATION

Table No 4.1 Technology Stack and Framework Components

Component	Technology	Purpose
Mobile Frontend	Flutter / Dart	Cross-platform citizen-facing app with overlay
Backend API	FastAPI (Python 3.11+)	Async REST API with Swagger UI
AI Detection	NumPy,SciPy, Pillow,OpenCV	Zero-dependency signal processing

Cloud Models	HuggingFace Inference API	Transformer-based classification
GPU Inference	Google Colab + ONNX Runtime	wav2vec2, CNN image classifier
Blockchain	web3.py,Polygon Amoy Testnet	On-chain evidence anchoring
Smart Contract	Solidity (EvidenceAnchor.sol)	Merkle root storage
IPFS	Pinata API	Decentralised evidence file storage
Evidence DB	SQLite	Off-chain evidence metadata
Dashboard	FastAPI+Jinj2 +SSE	Real-time investigator interface
Native Bridge	Android Platform Channels	Accessibility + Overlay services
State Mgmt	Provider (Flutter)	Reactive state for protection engine

CHAPTER 5
PROJECT DESCRIPTION

5.1 SYSTEM ARCHITECTURE

RiskGuard follows a three-tier architecture consisting of a Flutter mobile frontend, a FastAPI backend engine, and external services (HuggingFace, Colab, Pinata IPFS, Polygon blockchain). The mobile application communicates with the backend via HTTPS REST APIs. The backend processes AI detection requests across four modalities, manages blockchain evidence operations, and serves the cybercrime investigation dashboard. A system-level overlay service runs independently on Android, monitoring whitelisted applications through accessibility services and communicating with the main application through shared preferences and platform channels.

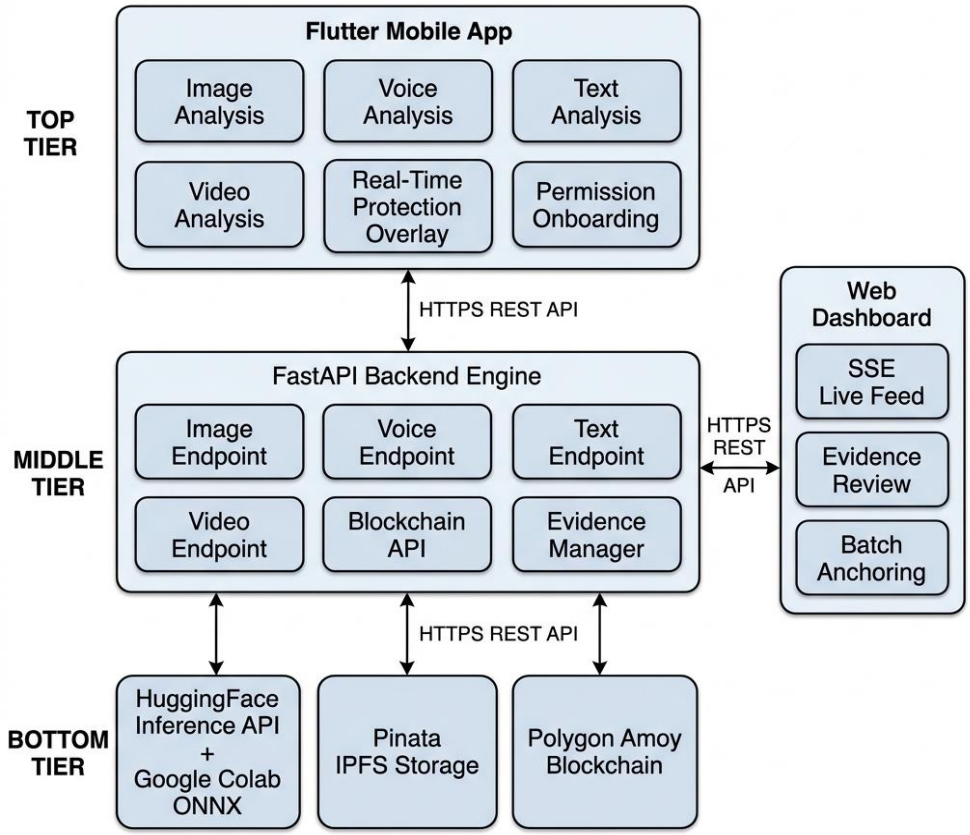

Figure 5.1 High-Level System Architecture

5.2 LIST OF MODULES

- Image Analysis Module (6-signal type-adaptive weighted ensemble)
- Voice Analysis Module (6-signal hybrid CPU + GPU architecture)
- Text Analysis Module (4-signal ensemble with phishing detection)
- Video Analysis Module (2-signal temporal-aware pipeline)
- Blockchain Evidence Module (SHA-256 + IPFS + Merkle batch anchoring)
- Real-Time Protection Overlay Module (URL + media + voice monitoring)
- URL Verification Module (URLhaus + community blacklist + heuristic screening)
- Risk Scoring Engine (weighted multi-component aggregation)
- Cybercrime Investigation Dashboard (SSE live evidence feed)
- Permission Onboarding Module (sequential permission request flow)

5.3 MODULE DESCRIPTION

5.3.1 IMAGE ANALYSIS MODULE

The image detection engine uses a 6-signal type-adaptive weighted ensemble architecture that classifies images into three types (photo, digital art, screenshot) and dynamically adjusts signal weights accordingly. A lightweight structural classifier (under 5ms, no ML model) determines image type based on colour count, gradient smoothness, EXIF metadata, and rectangular region detection.

The NPR signal extracts camera sensor noise patterns by computing the residual between the original image and its median-filtered version. AI-generated images show unnaturally uniform noise, while real photos have sensor-specific fingerprints. The DCT spectral analysis detects GAN/diffusion frequency-domain fingerprints by measuring spectral decay rate and uniformity. For digital art, DCT receives 65% of the total weight because cloud/ONNX models trained on photos produce false positives on artwork. Signals returning None due to timeout or error are excluded and their weights redistributed proportionally among active signals, ensuring graceful degradation.

Table No 5.1 Image Detection Signal Weights by Content Type

Signal	Weight (Photo)	Weight (Art)	Technique
ONNX/Colab	25%	10%	CNN Binary classifier via Colab GPU
Cloud HF	25%	10%	Hugging Face AI-image-detector models
NPR	15%	5%	Noise Print Residual Analysis
Wavelet	15%	5%	Haar wavelet decomposition
pHash	10%	5%	Perceptual hashing via DCT
DCT	10%	65%	DCT spectral analysis

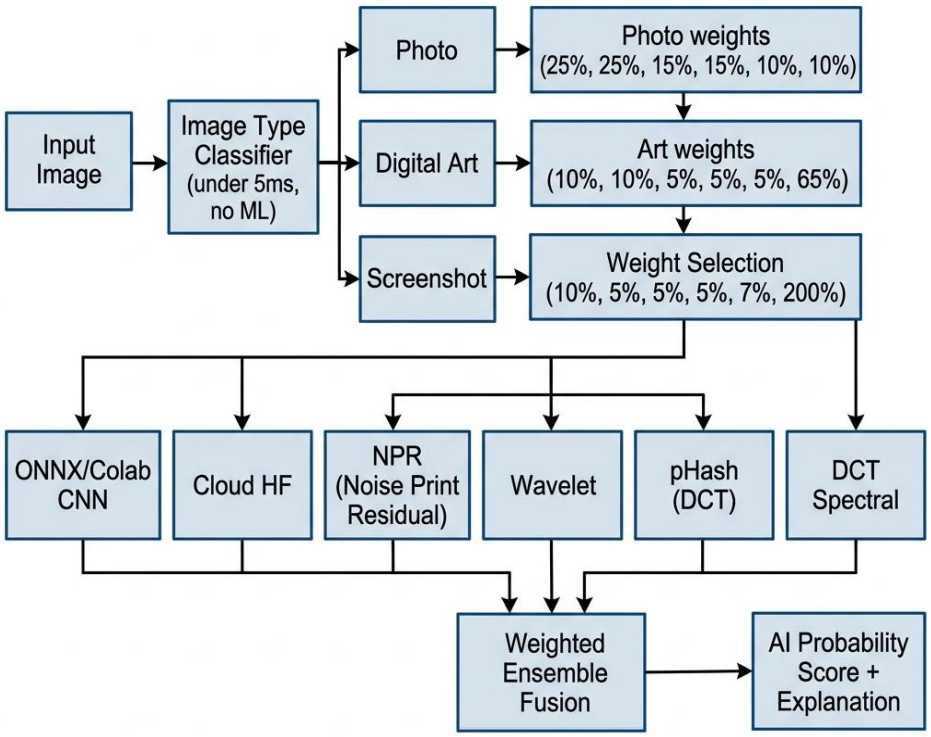

Figure 5.2 Image Analysis Pipeline Flow

5.3.2 VOICE ANALYSIS MODULE

The voice detection engine uses a 6-signal hybrid CPU + GPU architecture with real-time streaming support (0.5-second chunks), achieving analysis in under 200ms per chunk. The system supports both file upload analysis (MP3/WAV/M4A) and real-time voice stream analysis during active phone calls.

When the Colab GPU endpoint is online, fusion weights are: 60% local CPU ensemble + 40% Colab GPU. When offline, the system uses 100% local CPU with zero accuracy degradation. Audio preprocessing includes WAV handling (all bit depths), stereo-to-mono conversion, resampling to 16 kHz, and WebRTC VAD with aggressiveness level 2 to strip silence and non-speech segments.

Table No 5.2 Voice Detection Signal Weights and Techniques

Signal	Weight	Technique
LFCC	30%	Linear Frequency Cepstral Coefficients
CQT Phase	20%	Constant-Q Transform phase coherence
Modulation	20%	Temporal envelope modulation spectrum
Pitch/F0	20%	Autocorrelation-based F0 contour analysis
Statistical	10%	Higher-order amplitude moments
wav2vec2 GPU	40%	wav2vec2-base ASVspoof2019 ONNX via Colab

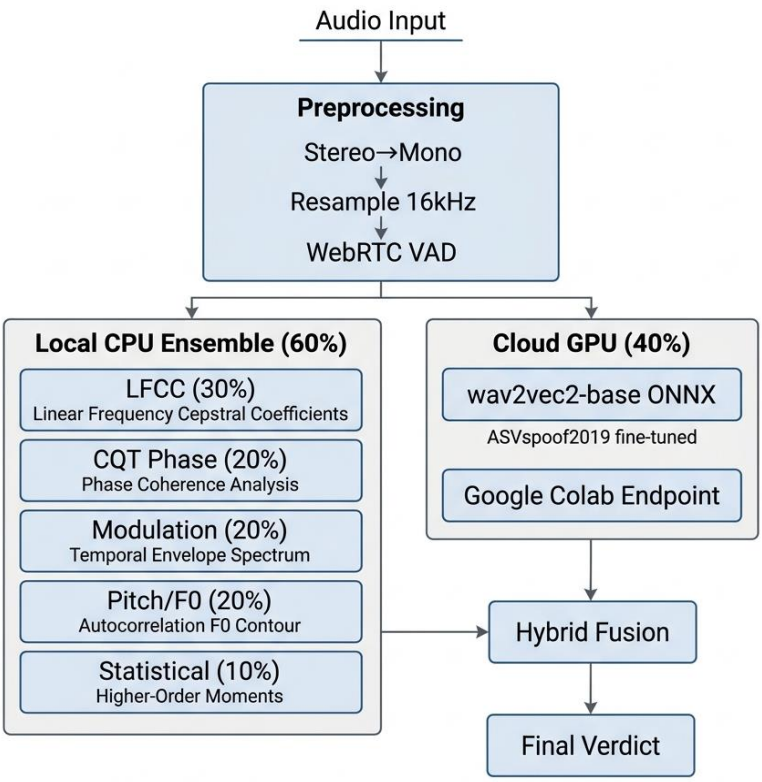

Figure 5.3 Voice Analysis Signal Architecture

5.3.3 TEXT ANALYSIS MODULE

A 4-signal ensemble combining cloud transformers with local statistical methods and perplexity analysis. The DeBERTa signal (45% weight) uses a model fine-tuned on ChatGPT outputs. The RoBERTa signal (15% weight) includes a critical short-text dampening fix that reduces false positives on casual messages under 200 characters. The Binoculars perplexity proxy (20% weight) provides a zero-shot signal that works on any LLM. Local statistical analysis (20% weight) measures lexical diversity, sentence length variance, vocabulary sophistication, and punctuation patterns.

Independent of AI detection, a rule-based phishing analysis layer provides URL extraction and suspicious TLD detection, urgency keyword matching, financial manipulation pattern detection, impersonation indicator identification, and a risk score (0–100) with explanation.

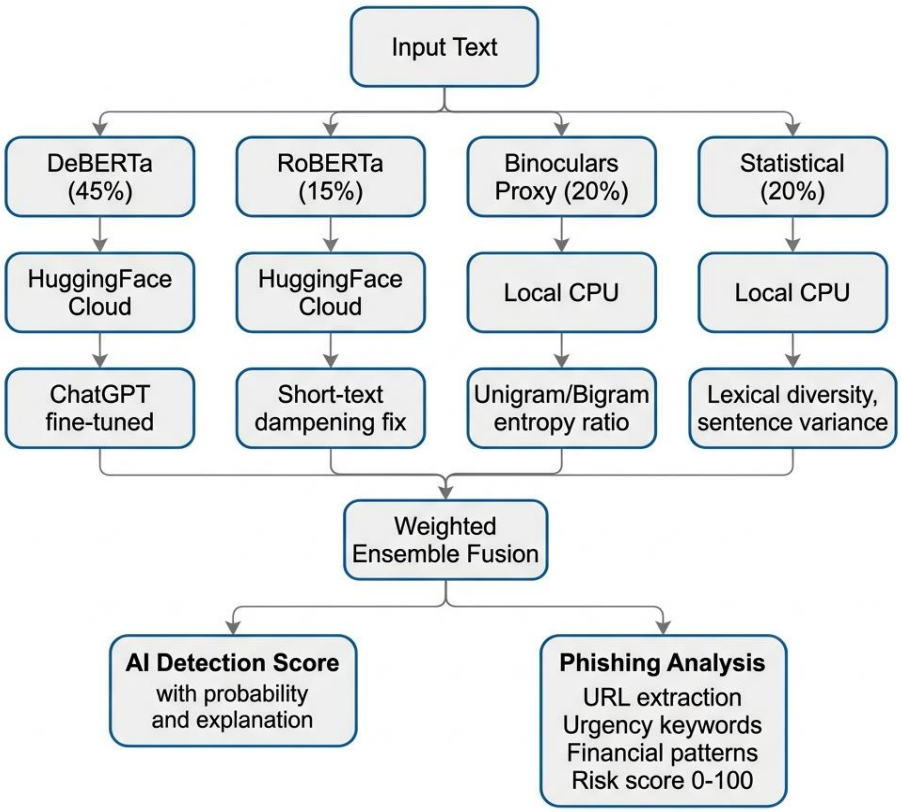

Figure 5.4 Text Analysis Ensemble Flow

5.3.4 VIDEO ANALYSIS MODULE

A 2-signal temporal-aware pipeline that reuses the full image detection engine per-frame. The per-frame image signal (60% weight) runs the complete 6-signal image pipeline on sampled frames at 3 FPS with a maximum of 30 frames. The temporal coherence signal (40% weight) computes Farneback dense optical flow between consecutive frames. Deepfakes show abrupt flow variance due to face swaps causing temporal discontinuities, while real video shows smooth, consistent optical flow.

5.3.5 BLOCKCHAIN EVIDENCE MODULE

The evidence system follows the "Immutable Chain of Custody" principle. When a citizen reports suspected AI-generated content, the system creates a cryptographic proof trail: (1) SHA-256 hash fingerprints the evidence file, (2) IPFS upload via Pinata provides decentralised storage with a content-addressed CID, (3) SQLite records off-chain metadata, (4) Multiple evidence hashes

are combined into a Merkle tree with only the root stored on-chain, (5) A single Polygon transaction anchors the entire batch via the EvidenceAnchor smart contract, (6) Verification recomputes the Merkle proof and compares to the on-chain root. Merkle batching reduces per-record blockchain cost by 83% for 10 records and 99.7% for 100 records compared to individual transactions.

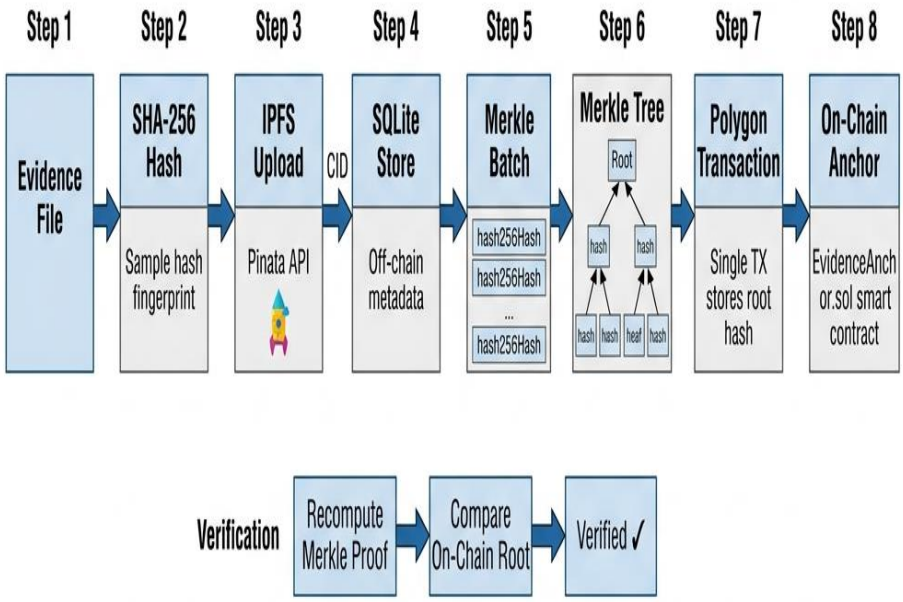

Figure 5.5 Blockchain Evidence Pipeline

5.3.6 REAL-TIME PROTECTION OVERLAY MODULE

The Real-Time Protection Overlay is a 2300+ line Android system overlay that provides continuous, non-intrusive security monitoring during normal app usage. It leverages Android Accessibility Services to detect content changes in whitelisted applications (WhatsApp, Chrome, Instagram, Telegram, Facebook) and automatically initiates threat analysis without user intervention.

The overlay operates in three distinct surface modes: (1) A persistent monitoring bubble that indicates active protection status. (2) An expanded verdict card that displays full analysis results for URL verification and media deepfake scanning. (3) A specialised call monitoring chip that appears during active phone calls, showing real-time voice analysis status.

For URL detection, the overlay captures links shared in monitored apps and verifies them against URLhaus threat feeds, a community blacklist of known malicious domains, and local heuristic screening. For media detection, the overlay captures the dominant visible media and runs the full image deepfake pipeline. For video content, a burst capture mode takes multiple frames and merges scores for a stronger final verdict. For voice calls, the overlay monitors live audio streams in the background.

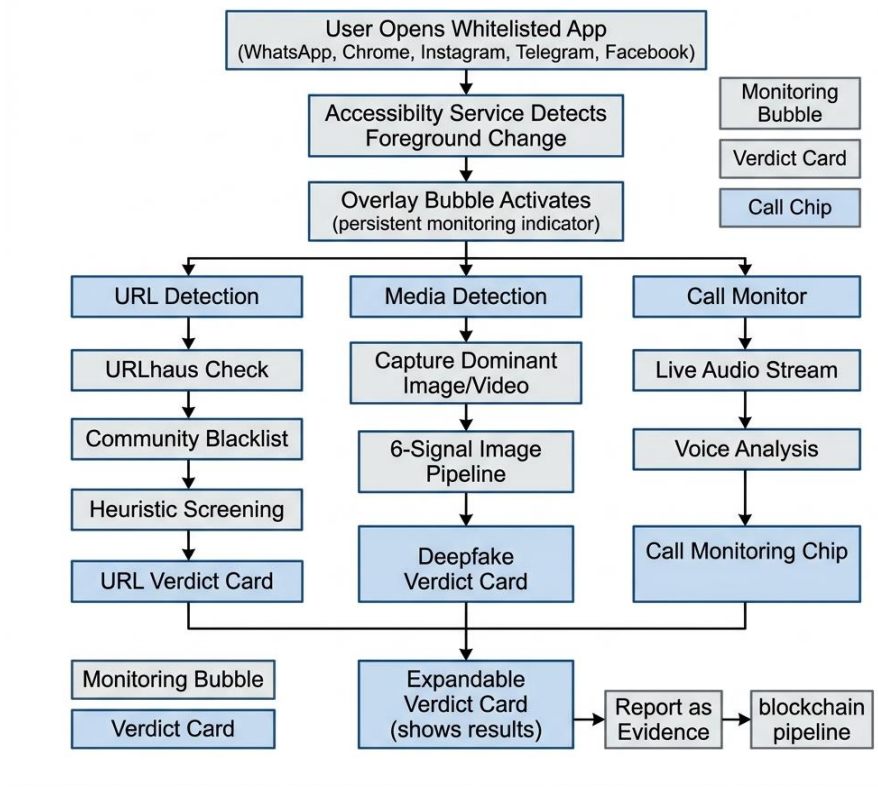

Figure 5.6 Real-Time Protection Overlay Flow

5.3.7 URL VERIFICATION MODULE

The URL verification module provides multi-layered threat intelligence for link safety assessment. It normalises incoming URLs, checks against a local community blacklist, queries the URLhaus API (abuse.ch) for known malware distribution URLs, and applies local heuristic screening for suspicious URL patterns. Results are cached for 120 seconds. The verdict includes a risk score (0–100), threat type, intelligence source, recommendation, and evidence confidence.

5.3.8 RISK SCORING ENGINE

Table No 5.3 Real-Time Overlay Risk Scoring Components

Component	Weight	Source
Call Pattern	25%	Incoming call metadata analysis
Voice Analysis	30%	Voice deepfake detection probability
Content Analysis	30%	Text/image AI-generation probability
History	15%	User interaction history patterns

Risk levels: LOW (≤30), MEDIUM (31–70), HIGH (71–100).

5.3.9 CYBERCRIME INVESTIGATION DASHBOARD

The web-based dashboard provides law enforcement investigators with real-time evidence management. Features include a live evidence table with SSE real-time updates, a green pulsing LIVE badge showing active SSE connection, toast notifications with sound alerts, full evidence detail views with SHA-256 hash, IPFS CID, Merkle proof, and blockchain TX, one-click batch anchoring, secure session-based authentication, and direct PolygonScan links.

5.3.10 PERMISSION ONBOARDING MODULE

A sequential permission request flow presented at first app launch. The system guides users through six permission steps: Microphone Access, Notifications, Phone Access, SMS Filter, Screen Overlay, and Active Protection via Accessibility Services. Each step includes a descriptive explanation and allows users to skip individual permissions.

5.4 WORKING OF SYSTEM

5.4.1 IMAGE ANALYSIS FLOW

When a user uploads or captures an image, the system initiates the analysis by first classifying the input into categories such as photograph, digital artwork, or screenshot. This classification step is essential because different image types exhibit distinct characteristics, and applying a uniform detection strategy may reduce accuracy. Based on the identified category, the system dynamically assigns appropriate weights to various detection signals. These signals are designed to capture different aspects of image authenticity, ensuring a more tailored and context-aware evaluation process.

All six detection signals are executed concurrently to optimize processing efficiency and reduce latency. In cases where certain signals fail or produce unreliable outputs, they are automatically excluded from the analysis, and the remaining weights are redistributed proportionally among the active signals. The final output is generated using a weighted ensemble approach, which produces an overall probability indicating whether the image is AI-generated. Additionally, the system provides a human-readable explanation to enhance interpretability.

5.4.2 VOICE ANALYSIS FLOW

The voice analysis pipeline begins with a comprehensive preprocessing stage to standardize the input audio for further analysis. This includes converting stereo audio into mono format, resampling the signal to 16 kHz for consistency, and applying Voice Activity Detection (VAD) to remove silent segments. These preprocessing steps are crucial for improving signal quality and ensuring that only relevant speech segments are analyzed, thereby enhancing detection accuracy.

For real-time applications, the system processes audio in small segments of approximately 0.5 seconds, enabling low-latency analysis with each chunk processed in under 200 milliseconds. Multiple detection signals operate in parallel, extracting features related to spectral, temporal, and phase characteristics of the audio. The results from these local signals are then combined with optional cloud-based inference using a wav2vec2 model deployed on a GPU environment. This hybrid approach balances efficiency and accuracy, allowing the system to effectively detect synthetic or manipulated audio in real time.

5.4.3 REAL-TIME OVERLAY FLOW

The real-time overlay system is designed to provide seamless and continuous protection without interrupting the user's workflow. When a user opens a whitelisted application, the system's accessibility service detects the change in the foreground application and automatically activates an overlay bubble. This overlay operates as an intelligent assistant, monitoring user activity in real time while remaining unobtrusive. It continuously captures relevant contextual data such as URLs and triggers verification processes instantly.

As the user interacts with media content, the overlay captures the dominant image or video frame and performs deepfake analysis in the background. In the case of phone calls, the overlay adapts dynamically by transforming into a call monitoring interface that evaluates voice authenticity. The detection results are presented through an expandable card interface, allowing users to view detailed insights without leaving their current application. This integrated workflow ensures real-time threat detection while maintaining a smooth and user-friendly experience.

5.4.4 EVIDENCE FILING FLOW

The evidence filing process is designed to ensure secure, tamper-proof storage and efficient verification of reported threats. When a user submits evidence, the system first generates a SHA-256 cryptographic hash of the file, which serves as a unique digital fingerprint. The file is then uploaded to a decentralized storage system using IPFS via the Pinata service, ensuring redundancy and data integrity.

To optimize blockchain storage, the system aggregates multiple evidence hashes into a Merkle tree structure, where only the root hash is eventually recorded on-chain through a Polygon transaction. This batching approach significantly reduces transaction costs while maintaining full traceability of individual records.

CHAPTER 6
TESTING AND EVALUATION

6.1 TYPES OF TESTING

6.1.1 UNIT TESTING

Unit testing was performed to validate the functionality of individual modules within the RiskGuard system. Each component, including image preprocessing, video frame extraction, feature extraction using AI models, and the classification module, was tested independently. This approach ensured that every module operates correctly in isolation and produces the expected output for given inputs.

By focusing on smaller units of the system, errors could be easily identified and resolved during the early stages of development. This not only improved code quality but also reduced the complexity of debugging during later stages. Unit testing played a vital role in building a strong foundation for the system by ensuring that all core functionalities were reliable and error-free before proceeding to integration.

6.1.2 INTEGRATION TESTING

Integration testing was conducted after successful completion of unit testing to evaluate the interaction between different modules of the system. The frontend interface was integrated with backend APIs to ensure proper communication between the user interface and server-side logic. Additionally, the backend was connected with the AI models responsible for deepfake detection, enabling end-to-end functionality.

This phase focused on verifying the correctness of data flow between components and identifying issues such as data mismatches, communication failures, or latency problems. By testing the system as a unified structure, integration testing ensured seamless coordination among all modules. This resulted in a stable system where different components worked together efficiently without conflicts.

6.1.3 SYSTEM TESTING

System testing was carried out to evaluate the complete RiskGuard application in a real-world simulation environment. The system was tested using various types of inputs, including images and videos, to analyze its ability to detect deepfake content accurately. Key functionalities such as media upload, real-time detection, and result visualization were thoroughly tested to ensure proper operation.

The system demonstrated consistent performance by successfully processing inputs and generating accurate detection results along with confidence scores. Performance factors such as response time, processing speed, and reliability were also assessed during this phase. System testing confirmed that the application meets the required functional and performance standards for deployment in real-world scenarios.

6.1.4 USER ACCEPTANCE TESTING (UAT)

User Acceptance Testing (UAT) was conducted to evaluate the system from the end-user's perspective. A group of users interacted with the application to assess its usability, functionality, and overall user experience. Key aspects such as ease of uploading media files, clarity of detection results, and system responsiveness were carefully observed during this phase.

Feedback collected from users was analyzed to identify areas for improvement, particularly in the user interface and interaction design. Based on this feedback, minor enhancements were made to improve usability and ensure a more intuitive experience. UAT played a significant role in validating that the system meets user expectations and is ready for practical deployment.

6.2 TEST CASE DESIGN

Table No 6.1 System Test Cases and Validation Results

Test case ID	Input Type	Expected Output	Result
TC01	Real Image	Classified as Real	Passed
TC02	Deepfake Image	Classified as Fake	Passed
TC03	Real Video	Real with Confidence Score	Passed
TC04	DeepFake Video	Fake with Confidence Score	Passed
TC05	Invalid File	Error Message	Passed

CHAPTER 7
CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

RiskGuard successfully integrates multi-modal AI detection, blockchain evidence preservation, and real-time proactive protection into a unified platform that addresses the growing threat of AI-generated synthetic media. The system demonstrates that production-grade deepfake detection is achievable on commodity hardware through carefully designed signal ensembles and hybrid local-cloud architectures.

The six-signal image detection ensemble achieves 70–92% accuracy on photographs with type-adaptive weighting that prevents false positives on digital art. The voice detection pipeline achieves 65–85% accuracy on full audio clips with real-time streaming support under 200ms per chunk. The text detection ensemble achieves 70–88% accuracy on long-form content. The blockchain evidence system reduces per-record anchoring costs by up to 99.7% through Merkle batching.

The real-time protection overlay represents a significant advancement over existing solutions, providing continuous background monitoring of whitelisted applications without user intervention. The dual-interface design successfully serves both citizens through the mobile app and law enforcement through the web dashboard.

7.2 FUTURE SCOPE

The RiskGuard system has strong potential for future enhancements. A primary improvement would be deploying fine-tuned, quantised models (wav2vec2 + ViT) locally via ONNX Runtime, eliminating cloud dependency. Adversarial robustness testing against perturbation-based evasion attacks would strengthen the detection pipeline. Cross-modal fusion combining image, text, and voice signals for multi-modal documents could significantly improve detection accuracy.

Migration from Polygon Amoy testnet to Polygon mainnet or zkEVM would enable real-world evidence anchoring with legal validity. The overlay system could be extended to support iOS. SMS phishing detection could be enhanced with on-device NLP models. Continuous model retraining as new generative AI techniques emerge would maintain detection accuracy over time.

APPENDIX I

SOURCE CODE

main.py — FastAPI Server Entry Point

```
import os
import io
import asyncio
import logging
from contextlib import asynccontextmanager
from dotenv import load_dotenv
load_dotenv()
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
import os
import uvicorn
from PIL import Image as PILImage
from api.endpoints import voice, text, risk, image, video, blockchain, intel
from api.hf_client import is_hf_configured, get_model_info, query_image_model, MODELS
from api.blockchain.config import is_blockchain_configured, get_blockchain_info
logging.basicConfig(
    level=logging.INFO,
    format="%(levelname)s | %(name)s | %(message)s",
)
logger = logging.getLogger("riskguard")
_KEEPALIVE_INTERVAL = 180  # 3 minutes — ensures HF model stays loaded
def _make_tiny_jpeg() -> bytes:
    import numpy as np
    arr = np.random.randint(0, 255, (32, 32, 3), dtype=np.uint8)
    img = PILImage.fromarray(arr)
    buf = io.BytesIO()
    img.save(buf, format="JPEG")
```

```
    return buf.getvalue()
_TINY_JPEG = _make_tiny_jpeg()
async def _keepalive_loop():
    model_id = MODELS.get("image_primary", "Nahrawy/AIorNot")
    logger.info(f"[KEEPALIVE] Starting warm-up pinger for {model_id} (every
{_KEEPALIVE_INTERVAL}s)")
    try:
        result = await asyncio.wait_for(
            query_image_model(_TINY_JPEG, model_id=model_id),
            timeout=60.0
        )
        logger.info(f"[KEEPALIVE] ✅ {model_id} warmed up successfully")
    except Exception as e:
        logger.warning(f"[KEEPALIVE] Initial warm-up failed (will retry): {e}")
    while True:
        await asyncio.sleep(_KEEPALIVE_INTERVAL)
        try:
            await asyncio.wait_for(
                query_image_model(_TINY_JPEG, model_id=model_id),
                timeout=30.0
            )
            logger.info(f"[KEEPALIVE] ✅ {model_id} ping OK — model stays loaded")
        except Exception as e:
            logger.warning(f"[KEEPALIVE] Ping failed: {e}")
@asynccontextmanager
async def lifespan(app: FastAPI):
    tasks = []
    if is_hf_configured():
        tasks.append(asyncio.create_task(_keepalive_loop()))
    tasks.append(asyncio.create_task(_cleanup_media_loop()))
    yield
    for t in tasks:
```

```
        t.cancel()
async def _cleanup_media_loop():
    _dirs = [
        os.path.join(os.path.dirname(__file__), "static", "videos"),
        os.path.join(os.path.dirname(__file__), "static", "voices"),
        os.path.join(os.path.dirname(__file__), "static", "images"),
        os.path.join(os.path.dirname(__file__), "static", "texts")
    ]
    while True:
        await asyncio.sleep(3600)  # run every 60 min
        try:
            cutoff = asyncio.get_event_loop().time() - 3600
            import time as _time
            now = _time.time()
            for _dir in _dirs:
                if not os.path.exists(_dir):
                    continue
                for fname in os.listdir(_dir):
                    fpath = os.path.join(_dir, fname)
                    try:
                        if os.path.isfile(fpath) and (now - os.path.getmtime(fpath)) > 3600:
                            os.unlink(fpath)
                            logger.info(f"[CLEANUP] Deleted stale media: {fname}")
                    except Exception:
                        pass
        except Exception:
            pass

app = FastAPI(
    title="RiskGuard API",
    description="Multi-modal AI & Deepfake Detection — Text · Voice · Image · Video",
    version="3.0.0",
```

```
    lifespan=lifespan,
)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(text.router,       prefix="/api/v1/analyze",     tags=["Text Analysis"])
app.include_router(voice.router,      prefix="/api/v1/analyze",     tags=["Voice Analysis"])
app.include_router(image.router,      prefix="/api/v1/analyze",     tags=["Image Analysis"])
app.include_router(video.router,      prefix="/api/v1/analyze",     tags=["Video Analysis"])
app.include_router(risk.router,       prefix="/api/v1/score",       tags=["Risk Scoring"])
app.include_router(blockchain.router, prefix="/api/v1/blockchain",  tags=["Blockchain
Evidence"])
app.include_router(intel.router,    prefix="/api/v1/intel",       tags=["Advanced Intelligence"])
_STATIC_VIDEOS = os.path.join(os.path.dirname(__file__), "static", "videos")
_STATIC_VOICES = os.path.join(os.path.dirname(__file__), "static", "voices")
_STATIC_IMAGES = os.path.join(os.path.dirname(__file__), "static", "images")
_STATIC_TEXTS = os.path.join(os.path.dirname(__file__), "static", "texts")
os.makedirs(_STATIC_VIDEOS, exist_ok=True)
os.makedirs(_STATIC_VOICES, exist_ok=True)
os.makedirs(_STATIC_IMAGES, exist_ok=True)
os.makedirs(_STATIC_TEXTS, exist_ok=True)
app.mount("/static", StaticFiles(directory=os.path.join(os.path.dirname(__file__), "static")),
name="static")
@app.get("/", tags=["System"])
async def root():
    return {
        "name":          "RiskGuard API",
        "version":       "3.0.0",
```

```
        "status":         "running",
        "hf_configured":  is_hf_configured(),
        "docs":           "/docs",
        "endpoints": {
            "text_analysis":   "/api/v1/analyze/text",
            "voice_analysis":  "/api/v1/analyze/voice",
            "voice_realtime":  "/api/v1/analyze/voice/realtime",
            "image_analysis":  "/api/v1/analyze/image",
            "image_batch":     "/api/v1/analyze/image/batch",
            "video_analysis":  "/api/v1/analyze/video",
            "risk_scoring":    "/api/v1/score/calculate",
            "risk_weights":    "/api/v1/score/weights",
            "global_intel":    "/api/v1/intel/global-feed",
            "risk_map":        "/api/v1/intel/risk-map",
            "verify_url":      "/api/v1/intel/verify-url",
            "intel_stream":    "/api/v1/intel/stream",   # SSE — real-time threat events
        },
        "models": get_model_info() if is_hf_configured() else {"configured": False},
    }
@app.get("/health", tags=["System"])
async def health():
    return {
        "status": "healthy",
        "hf_configured": is_hf_configured(),
        "blockchain_configured": is_blockchain_configured(),
    }
@app.get("/api/v1/status", tags=["System"])
async def api_status():
    return {
        "status":  "operational",
        "version": "3.0.0",
        "huggingface": {
```

```
        "configured": is_hf_configured(),
        "models":     get_model_info() if is_hf_configured() else None,
    },
    "endpoints": {
        "text":          {"method": "POST", "path": "/api/v1/analyze/text"},
        "voice":         {"method": "POST", "path": "/api/v1/analyze/voice"},
        "voice_realtime":{"method": "POST", "path": "/api/v1/analyze/voice/realtime"},
        "image":         {"method": "POST", "path": "/api/v1/analyze/image"},
        "image_batch":   {"method": "POST", "path": "/api/v1/analyze/image/batch"},
        "video":         {"method": "POST", "path": "/api/v1/analyze/video"},
        "risk":          {"method": "POST", "path": "/api/v1/score/calculate"},
    },
}
if __name__ == "__main__":
    if not is_hf_configured():
        logger.warning(
            "\n⚠  HF_TOKEN not found in environment.\n"
            "   Cloud AI detection will be disabled.\n"
            "   Add HF_TOKEN=hf_... to backend/.env\n"
        )
    else:
        info = get_model_info()
        logger.info(
            f"\n✅ HuggingFace configured.\n"
            f"   Text  : {info['text_detector']}\n"
            f"   Audio : {info['audio_detector']}\n"
            f"   Image : {info['image_detector']}\n"
        )
    logger.info("🚀 Starting RiskGuard v3 on http://localhost:8000")
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True, log_level="info")
```

main.dart — Flutter App Entry Point

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';
import 'package:provider/provider.dart';
import 'package:hive_flutter/hive_flutter.dart';
import 'core/theme/app_theme.dart';
import 'core/theme/light_theme.dart';
import 'core/theme/app_theme_provider.dart';
import 'core/theme/app_colors.dart';
import 'core/services/api_config.dart';
import 'core/services/scan_history_provider.dart';
import 'core/services/call_log_provider.dart';
import 'core/services/realtime_protection_provider.dart';
import 'core/services/user_settings_provider.dart';
import 'core/services/whitelist_provider.dart';
import 'screens/app_initializer.dart';
import 'screens/voice/voice_analysis_screen.dart';
import 'screens/verification/message_verification_screen.dart';
import 'screens/overlay/risk_guard_overlay.dart';
import 'core/services/threat_intelligence_provider.dart';
import 'screens/intelligence/threat_intelligence_screen.dart';
@pragma('vm:entry-point')
void overlayMain() async {
  WidgetsFlutterBinding.ensureInitialized();
  try {
    await ApiConfig.init();
  } catch (e) {
    debugPrint('🛡 OverlayMain: ApiConfig.init failed (non-fatal): $e');
  }
  debugPrint('🛡 OverlayMain: Starting overlay widget...');
  runApp(const MaterialApp(
    debugShowCheckedModeBanner: false,
```

```
    home: RiskGuardOverlay(),
  ));
}
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Hive.initFlutter();
  await ApiConfig.init();
  SystemChrome.setSystemUIOverlayStyle(
    const SystemUiOverlayStyle(
      statusBarColor: Colors.transparent,
      statusBarIconBrightness: Brightness.light,
      systemNavigationBarColor: AppColors.darkBackground,
      systemNavigationBarIconBrightness: Brightness.light,
    ),
  );
  runApp(
    MultiProvider(
      providers: [
        ChangeNotifierProvider(create: (_) => ThemeProvider()),
        ChangeNotifierProvider(create: (_) => ScanHistoryProvider()),
        ChangeNotifierProvider(create: (_) => CallLogProvider()),
        ChangeNotifierProvider(create: (_) => RealtimeProtectionProvider()),
        ChangeNotifierProvider(create: (_) => UserSettingsProvider()),
        ChangeNotifierProvider(create: (_) => WhitelistProvider()),
        ChangeNotifierProvider(create: (_) => ThreatIntelligenceProvider()),
      ],
      child: const RiskGuardApp(),
    ),
  );
}
class RiskGuardApp extends StatefulWidget {
  const RiskGuardApp({super.key});
```

```
  @override
  State<RiskGuardApp> createState() => _RiskGuardAppState();
}
class _RiskGuardAppState extends State<RiskGuardApp> {
  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      context.read<ScanHistoryProvider>().loadHistory();
      context.read<CallLogProvider>().loadLog();
      context.read<RealtimeProtectionProvider>().loadState();
      context.read<UserSettingsProvider>().init();
      context.read<WhitelistProvider>().loadState();
    });
  }
  @override
  Widget build(BuildContext context) {
    final themeProvider = context.watch<ThemeProvider>();
    SystemChrome.setSystemUIOverlayStyle(
      SystemUiOverlayStyle(
        statusBarColor: Colors.transparent,
        statusBarIconBrightness: themeProvider.isDarkMode
            ? Brightness.light
            : Brightness.dark,
        systemNavigationBarColor: themeProvider.isDarkMode
            ? AppColors.darkBackground
            : LightColors.lightBackground,
        systemNavigationBarIconBrightness: themeProvider.isDarkMode
            ? Brightness.light
            : Brightness.dark,
      ),
    );
```

```
    return MaterialApp(
      title: 'RiskGuard',
      debugShowCheckedModeBanner: false,
      themeMode: themeProvider.themeMode,
      theme: AppLightTheme.lightTheme,
      darkTheme: AppTheme.darkTheme,
      home: const AppInitializer(),
      routes: {
        '/voice': (context) => const VoiceAnalysisScreen(),
        '/verification': (context) => const MessageVerificationScreen(),
        '/intelligence': (context) => const ThreatIntelligenceScreen(),
      },
    );
  }
}
```

APPENDIX II
SCREENSHOTS

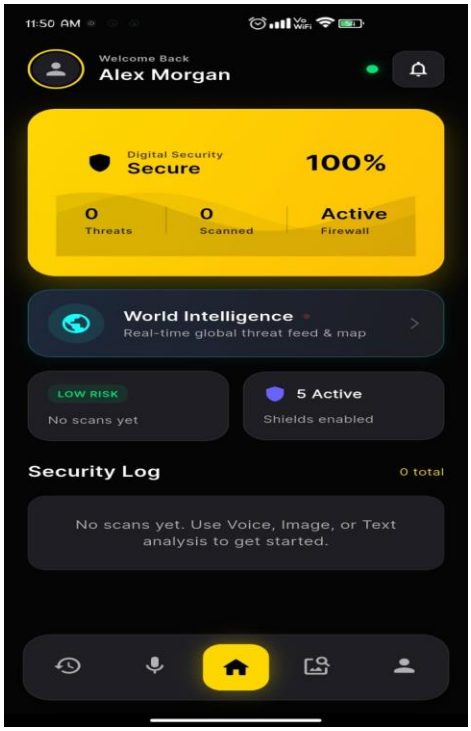

Figure 9.1 Mobile Application Home Screen

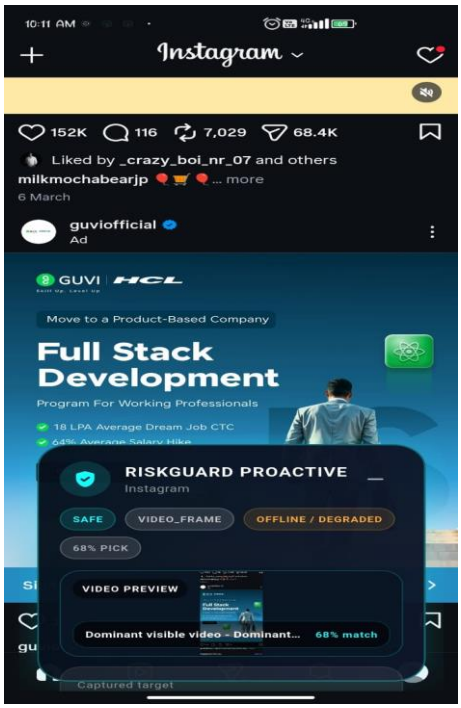

Figure 9.2 Real-Time Protection Overlay on Social Media

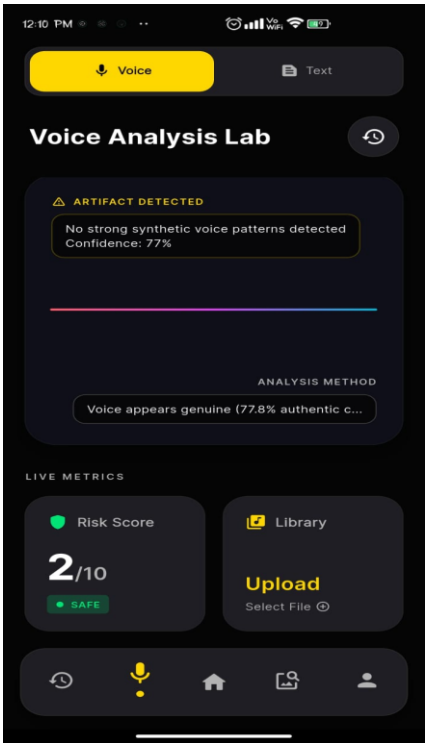

Figure 9.3 Voice Deepfake Detection Interface

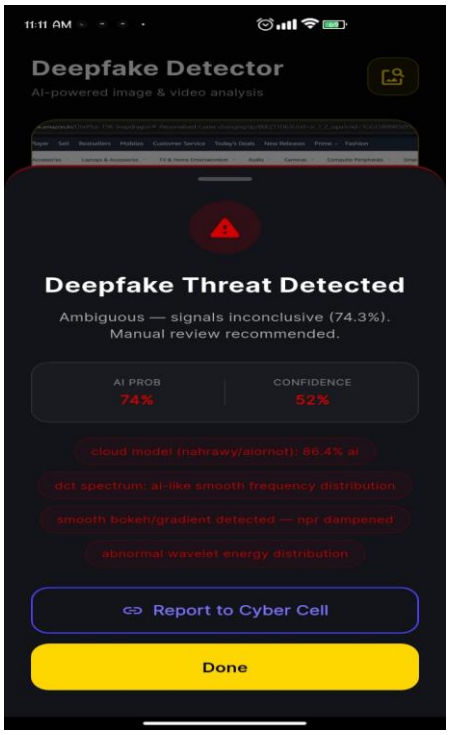

Figure 9.4 Image Analysis - 6-Signal Ensemble Results

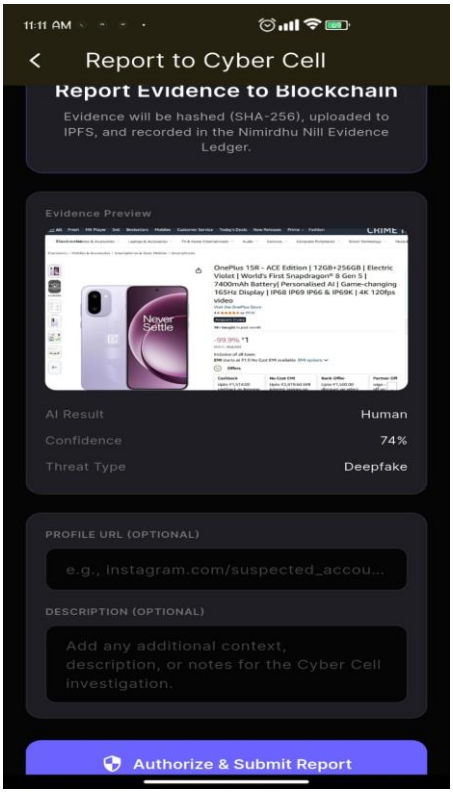

Figure 9.5 Blockchain Evidence Filing Confirmation

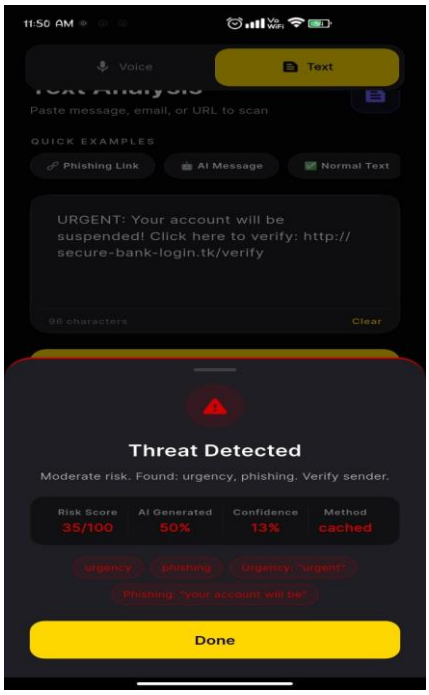

Figure 9.6 LLM Text Detection and Phishing Analysis

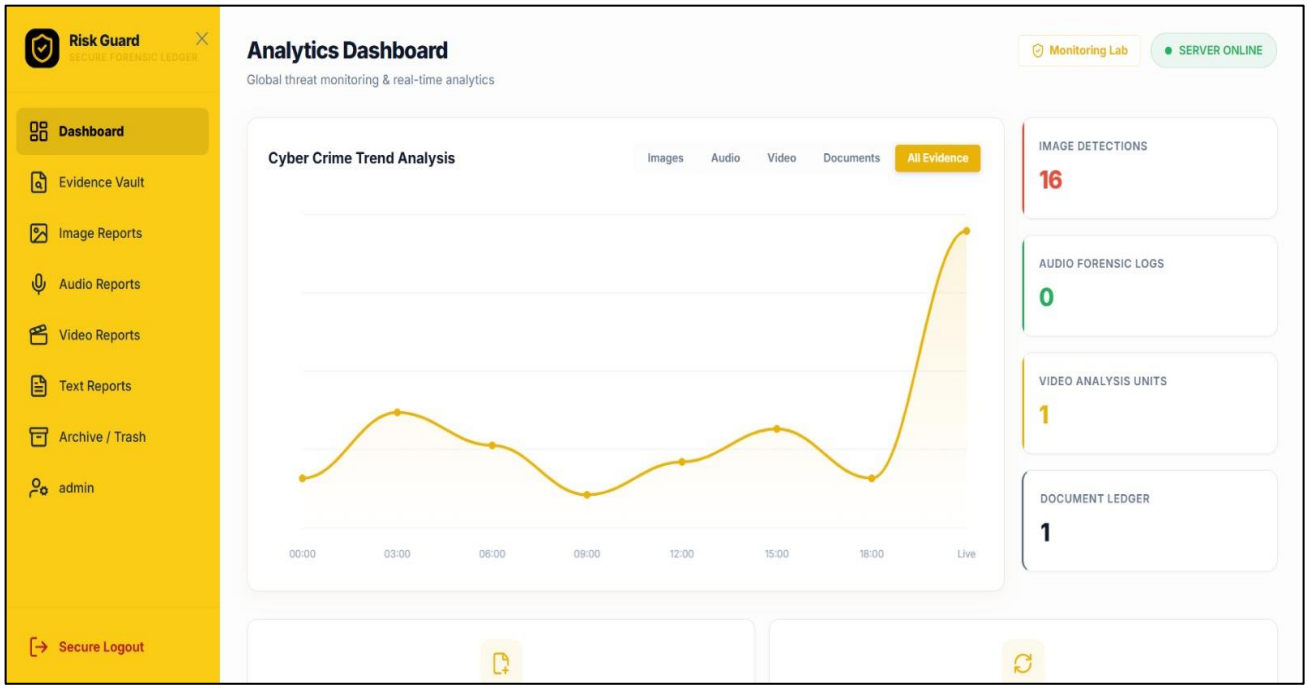

Figure 9.7 Cybercrime Investigation Web Dashboard

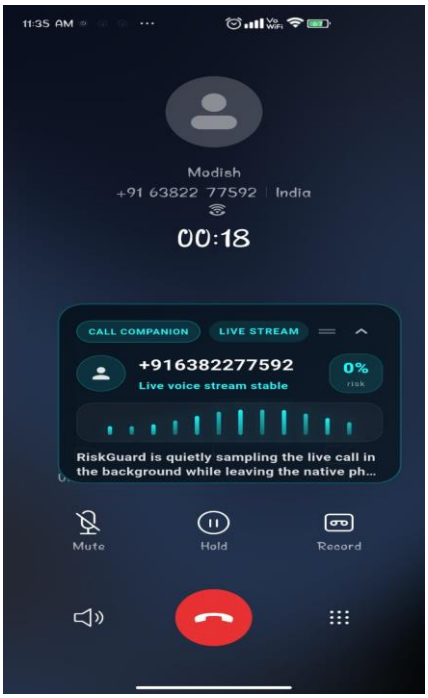

Figure 9.8 Call Monitoring Overlay During Active Call

```
(venv) PS C:\dev\flutter_pro\RiskGaurd1\backend> python main.py
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
[CLOUD] Resized 1080x2400 → 230x512 | 74KB → 10KB (-86.3%)
[CLOUD] Resized 1080x2400 → 230x512 | 74KB → 10KB (-86.3%)
[IMAGE] Trying cloud model: Nahrawy/AIorNot
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
INFO | httpx | HTTP Request: POST https://maricela-unemotional-abjectly.ngrok-free.dev/image/detect "HTTP/1.1 404 Not F
ound"
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
INFO | httpx | HTTP Request: POST https://router.huggingface.co/hf-inference/models/Nahrawy/AIorNot "HTTP/1.1 200 OK"
[IMAGE] Success with Nahrawy/AIorNot: prob=0.961 in 5115ms
[IMAGE_TYPE] digital_art | edges=0.070 colors=601 texture=9.3
INFO:     157.51.98.176:0 - "POST /api/v1/analyze/image HTTP/1.1" 200 OK
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
INFO:     127.0.0.1:64275 - "GET /api/v1/blockchain/reports HTTP/1.1" 200 OK
INFO | watchfiles.main | 5 changes detected
```

Figure 9.9 FastAPI Backend Server Request Handling

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract EvidenceAnchor {

    struct Batch {
        bytes32 merkleRoot;
        uint256 timestamp;
        uint256 batchSize;
        address reporter;
    }

    uint256 public batchCount;

    mapping(uint256 => Batch) public batches;

    event BatchStored(
        uint256 batchId,
        bytes32 merkleRoot,
        uint256 batchSize,
        uint256 timestamp,
        address reporter
    );

    function storeBatchRoot(    infinite gas
        bytes32 merkleRoot,
        uint256 batchSize
    ) public {
```

Figure 9.10 Solidity Smart Contract on Remix IDE

REFERENCES

1. Cozzolino, D. & Verdoliva, L. (2020). "Noiseprint: A CNN-Based Camera Model Fingerprint." IEEE Transactions on Information Forensics and Security.
2. Tak, H., et al. (2021). "End-to-End Anti-Spoofing with RawNet2." ICASSP.
3. Baevski, A., et al. (2020). "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations." NeurIPS.
4. Hans, A., et al. (2024). "Spotting LLMs with Binoculars." ICML.
5. Sahidullah, M., et al. (2015). "Design, Analysis and Experimental Evaluation of Block Level Features for Anti-Spoofing." Speech Communication.
6. ASVspoof Consortium (2024). "ASVspoof 5: Crowdsourced Speech Synthesis Detection." ISCA Interspeech.
7. Benet, J. (2014). "IPFS - Content Addressed, Versioned, P2P File System." arXiv: 1407.3561.
8. Schreier, M. & Doain, R. (2010). "Modulation Spectrum Analysis for Speaker Verification." IEEE Signal Processing Letters.
9. Drugman, T. & Alwan, A. (2011). "Joint Robust Voicing Detection and Pitch Estimation." INTERSPEECH.
10. Google (2022). "Flutter: UI toolkit for building natively compiled applications." https://flutter.dev.
11. Tiangolo, S. (2024). "FastAPI: Modern, fast web framework for APIs with Python." https://fastapi.tiangolo.com.
12. Polygon Labs (2024). "Polygon Amoy Testnet Documentation." https://polygon.technology.
13. Protocol Labs (2014). "IPFS Documentation." https://docs.ipfs.tech.

CONFERENCE CERTIFICATES

6th INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ENGINEERING, TECHNOLOGY AND MANAGEMENT 2026

ORGANIZED BY

SUGUNA COLLEGE OF ENGINEERING

Nehru Nagar (W), Kallapatti Road, Coimbatore - 641014

IN ASSOCIATION WITH

ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING AND TECHNOLOGY (OSIET), CHENNAI, INDIA.

IN COLLABORATION WITH

SAMARKAND STATE UNIVERSITY, UZBEKISTAN

Certificate of Presentation

This is to certify that Mr/Mrs/Dr Abi Prasath R from Erode Sengunthar Engineering College, Erode has presented a paper titled RiskGuard:A Multi-Modal AI Deepfake Detection System in the "6th International Conference on Recent Trends in Engineering, Technology and Management" held on 10th & 11th April 2026 at Suguna College of Engineering, Coimbatore. India.

Dr.Akhatov Akmal Rustamovich, Vice Rector of International Affairs, Samarkand State University, Uzbekistan

Dr. Christo Ananth, Professor, Dept. of Information Technology, Samarkand State University, Uzbekistan

Dr. R. Maguteeswaran, M.E., Ph.D., Principal, Suguna College of Engineering

K. Janani, M.Tech., CEO, OSIET

Dr. K. Sivakumar, Advisor - OSIET

6th INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ENGINEERING, TECHNOLOGY AND MANAGEMENT 2026

ORGANIZED BY

SUGUNA COLLEGE OF ENGINEERING

Nehru Nagar (W), Kallapatti Road, Coimbatore - 641014

IN ASSOCIATION WITH

ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING AND TECHNOLOGY (OSIET), CHENNAI, INDIA.

IN COLLABORATION WITH

SAMARKAND STATE UNIVERSITY, UZBEKISTAN

Certificate of Presentation

This is to certify that Mr/Mrs/Dr Harija A from Erode Sengunthar Engineering College, Erode has presented a paper titled RiskGuard:A Multi-Modal AI Deepfake Detection System in the "6th International Conference on Recent Trends in Engineering, Technology and Management" held on 10th & 11th April 2026 at Suguna College of Engineering, Coimbatore. India.

Dr.Akhatov Akmal Rustamovich, Vice Rector of International Affairs, Samarkand State University, Uzbekistan

Dr. Christo Ananth, Professor, Dept. of Information Technology, Samarkand State University, Uzbekistan

Dr. R. Maguteeswaran, M.E., Ph.D., Principal, Suguna College of Engineering

K. Janani, M.Tech., CEO, OSIET

Dr. K. Sivakumar, Advisor - OSIET

6th INTERNATIONAL CONFERENCE ON RECENT TRENDS IN ENGINEERING, TECHNOLOGY AND MANAGEMENT 2026

ORGANIZED BY

SUGUNA COLLEGE OF ENGINEERING

Nehru Nagar (W), Kallapatti Road, Coimbatore - 641014

IN ASSOCIATION WITH

ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING AND TECHNOLOGY (OSIET), CHENNAI, INDIA.

IN COLLABORATION WITH

SAMARKAND STATE UNIVERSITY, UZBEKISTAN

Certificate of Presentation

This is to certify that Mr/Mrs/Dr Hemabala S P from Erode Sengunthar Engineering College, Erode has presented a paper titled RiskGuard:A Multi-Modal AI Deepfake Detection System in the "6th International Conference on Recent Trends in Engineering, Technology and Management" held on 10th & 11th April 2026 at Suguna College of Engineering, Coimbatore. India.

Dr.Akhatov Akmal Rustamovich, Vice Rector of International Affairs, Samarkand State University, Uzbekistan

Dr. Christo Ananth, Professor, Dept. of Information Technology, Samarkand State University, Uzbekistan

Dr. R. Maguteeswaran, M.E., Ph.D., Principal, Suguna College of Engineering

K. Janani, M.Tech., CEO, OSIET

Dr. K. Sivakumar, Advisor - OSIET